UNIT-III

CONTROL FLOW, FUNCTIONS

Conditionals: Boolean values and operators, conditional (if), alternative (if-else), chained conditional (if-elif-else); Iteration: state, while, for, break, continue, pass; Fruitful functions: return values, parameters, local and global scope, function composition, recursion; Strings: string slices, immutability, string functions and methods, string module; Lists as arrays. Illustrative programs: square root, gcd, exponentiation, sum an array of numbers, linear search, binary search.

BOOLEAN VALUES AND OPERATORS:

Boolean values can be tested for truth value, and used for IF and WHILE condition. There are two values True and False. 0 is considered as False and all other values considered as True.

Boolean Operations:

Consider x=True, y= False

Operator	**Example**	**Description**
and	x and y- returns false	Both operand should be true
or	x or y- returns true	Anyone of the operand should be true
not	not x returns false	Not carries single operand

Boolean values are used in comparisons and conditional expressions. The most common way to produce boolean value is relational operators.

Operator	Description	Example
= =	Equal to	>>>4 = = 4 True
!=	Not equal to	>>> 4 != 3 True
>	Greater than	>>> 4 > 2 True
<	Less than	>>> 4< 2 False
>=	Greater than or equal to	>>>4 >= 5 False
<=	Less than or equal to	>>> 4<= 5 True

Decision Making (or) Conditionals (or) Branching

The execution of the program depends upon the condition. The sequence of the control flow differs from the normal program. The decision making statements evaluate the conditions and produce True or False as outcome.

Types of conditional Statement

1. if statement
2. if-else statement (alternative)
3. if-elif-else statement (chained conditional)

if Statement :

Syntax:

```
if test expression:
        statement(s)
```

- Here, the program evaluates the test expression and will execute statement(s) only if the text expression is True. If the text expression is False, the statement(s) is not executed.
- In Python, the body of the if statement is indicated by the indentation. Body starts with an indentation

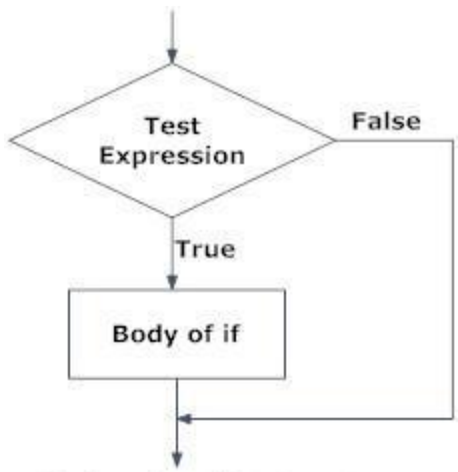

Fig: Operation of if statement

num = input(**"Enter a number: "**) **if** num > 0: **print**(**"Positive number"**)	Enter a number: 12 Positive number
num = input(**"Enter ur age: "**) **if** num < 20 : **print**(**"Ur a teenagers"**)	Enter ur age: 14 Ur a teenagers

Python if...else

Syntax

```
if test expression:
    Body of if
else:
    Body of else
```

- The if..else statement evaluates test expression and will execute body of if only when test condition is True. If the condition is False, body of else is executed. Indentation is used to separate the blocks.

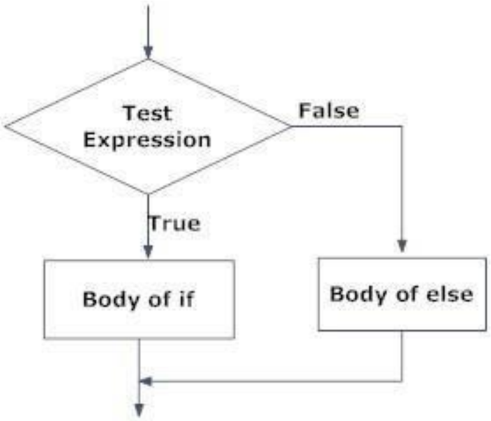

Fig: Operation of if...else statement

num = input(**"Enter a number: "**)
if num >= 0:
   print(**"Positive or Zero"**)
else:
   print(**"Negative number"**)
	Enter a number: 1
	Positive or Zero
	Enter a number: -2
	Negative number

Find the given no. is ODD or EVEN
 	
num = input(**"Enter the number:"**)
if (num % 2 == 1):
   print num,**' is ODD'**
else:
   print num, **'is Even'**
	Enter the number:90
	90 is Even

Python if...elif...else

Syntax

```
if test expression:
    Body of if
elif test expression:
    Body of elif
else:
    Body of else
```

- The elif is short for else if. It allows us to check for multiple expressions. If the condition for if is False, it checks the condition of the next elif block and so on. If all the conditions are False, body of else is executed.
- Only one block among the several if...elif...else blocks is executed according to the condition. A if block can have only one else block. But it can have multiple elif blocks.

Flowchart:

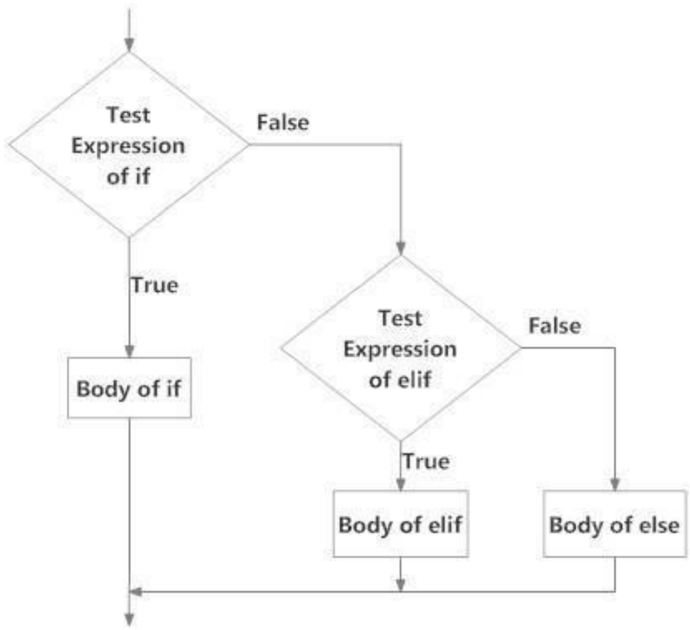

Fig: Operation of if...elif...else statement

Example:

num = input(**"Enter a number: "**)
if num > 0:
  print(**"Positive number"**)
elif num == 0:
  print(**"Zero"**)
else:
  print(**"Negative number"**)
	Enter a number: 7
	Positive number
	Enter a number: 0
	Zero
	Enter a number: -1
	Negative number

age = input(**"Enter the age: "**)
if age>0 **and** age<=5:
  print(**"Young Children"**)
elif age>5 **and** age<=19:
  print(**"Teanagers"**)
	Enter the age: 25
	Young Adults
	
	Enter the age: 45
	Younger Middle-aged Adults

elif age > 20 **and** age <= 34:
  print(**"Young Adults"**)
elif age > 35 **and** age <= 49:
  print(**"Younger Middle-aged Adults"**)
elif age > 50 **and** age <= 64:
  print(**"Older Middle-aged Adults"**)
else:
  print(**"Senior Adults"**)	

Eg:

```
num = 7
factorial = 1
if num < 0:
   print("Sorry, factorial does not exist for negative numbers")
elif num == 0:
   print("The factorial of 0 is 1")
else:
   for i in range(1,num + 1):
       factorial = factorial*i
   print("The factorial of",num,"is",factorial)
```

Python while Loop

- The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true. We generally use this loop when we don't know beforehand, the number of times to iterate.

Syntax of while Loop

```
while test_expression:
      Body of while
```

- In while loop, test expression is checked first. The body of the loop is entered only if the test_expression evaluates to True. After one iteration, the test expression is checked again. This process continues untill the test_expression evaluates to False

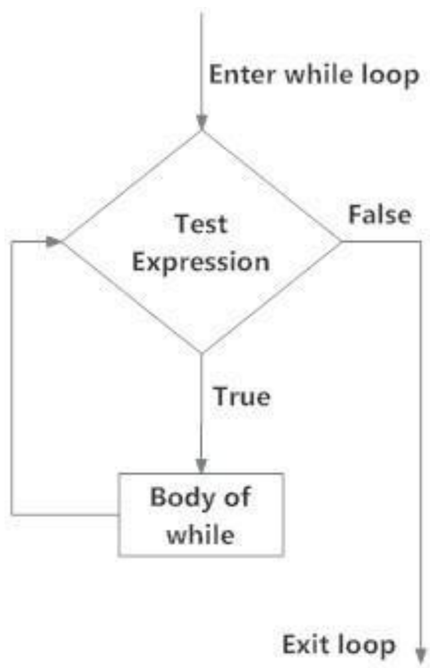

Fig: operation of while loop

a=0
while a<=10:
   print a
   a=a+1
	0 1 2 3 4 5 6 7 8 9 10

n=input("**Enter n :** ")
i=0
sum=0
while i<=n:
   sum=sum+i
   i=i+1
print "Sum =",sum
	Enter n : 10
	Sum = 55

for Loop

- For loop repeats a group of statements for a specified number of times. For loop starts with the keyword "for" followed by an arbitrary variable name, which holds its values in the following sequence object.

Syntax:

for <loop-variable> in <sequence>:
        Body of for

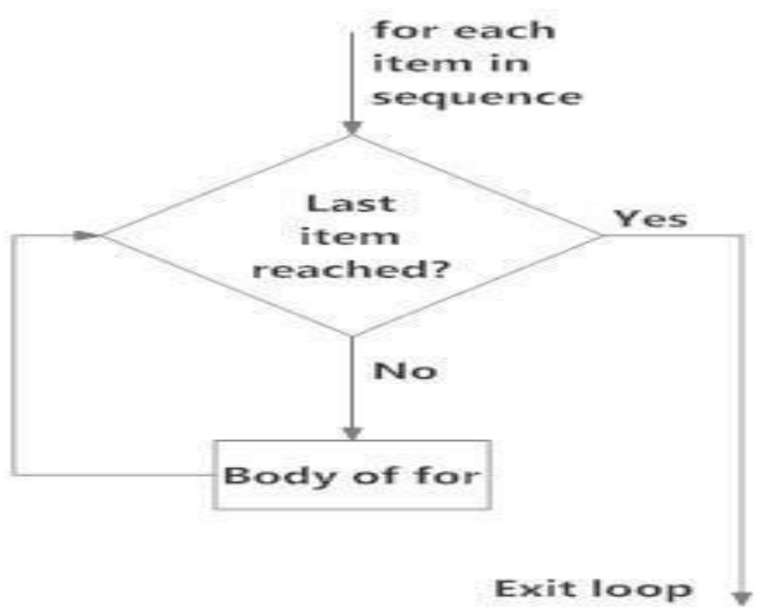

Fig: operation of for loop

The Range() function:

- Sequence of numbers are also generated using range() function. range() is defined with start_element, stop_element, and step size.
- Default step size is equal to 1 if not provided. The range () function is useful in combination with the for loop.

Syntax:

Range (start_element, stop_element, step size)

- Range(n) - generates numbers from 0 to n-1;
  eg: range(10)= 0 1 2 3 4 5 6 7 8 9
- Range(m,n) - generates numbers from m to n-1;
  eg: range(4,10)= 4 5 6 7 8 9
- Range(m,n,x) - generates numbers from m upto to n-1 with skip counting of x
  Eg: range(0,10,2)= 0 2 4 6 8

Example:

n={2,4,6,5,3,10} **for** i **in** n: **print** i	2 3 4 5 6 10
n=10 **for** i **in** range(n): **print** i	0 1 2 3 4 5 6 7 8 9
n=10 **for** i **in** range(4,n): **print** i	4 5 6 7 8 9
n=10 **for** i **in** range(1,n,2): **print** i	1 3 5 7 9
n=input(**"Enter n : "**) sum=0 **for** i **in** range(1,n): sum=sum+i **print "Sum ="**,sum	Enter n : 10 Sum = 45
n={2,4,6,5,3,10} sum=0 **for** i **in** n: sum=sum+i **print "Sum ="**,sum	Sum = 30